1 Positive-rank multiplicative tensors

Source: `PrimeTensor/Tensor/Basic.lean`

What this file establishes

A tensor is a function from index tuples to coefficients, wrapped in a one-field structure. Both its dimension and its rank are depths, so neither can be zeroth: every tensor has at least one dimension and at least one index. The coefficient type is arbitrary, and the one operation that needs anything of it — the outer product — asks only for a multiplication, with no laws.

The file is short, and almost all of its content is that the definitions typecheck at all. That they do is inherited from `PrimeTensor/Depth.lean`: because IndexTuple was defined there by recursion into Type rather than as an inductive family, a rank-one index tuple *is* an axis index and a rank-two one *is* a pair, definitionally, with no coercion in between. Section 1.2 is where that is spent.

1.1 Tensors

Definition 1.1 (Tensor, `PrimeTensor.Tensor`)**.** For a type A and depths *dim*, *rank*, the structure Tensor A *dim rank* has the single field

$$\text{component} : \text{IndexTuple}(\mathit{dim}, \mathit{rank}) \to A.$$

```
structure Tensor (A : Type) (dim rank : Depth) where
  component : IndexTuple dim rank → A
```

Wrapping the function in a structure rather than using the function type directly buys nominal typing: Tensor A *dim rank* and $\text{IndexTuple}(\mathit{dim}, \mathit{rank}) \to A$ are different types, so a tensor cannot be applied by mistake and the elaborator will not unify a tensor with an unrelated function. Nothing is lost by it: Lean's structure eta makes $\langle f \rangle$.component and $\langle t.\text{component} \rangle$ reduce to f and t, so the wrapper is transparent in proofs.

Remark 1.2 (Size)*.* Tensor A *dim rank* is the type of functions from a finite set of $|\mathit{dim}|^{|\mathit{rank}|}$ indices to A, by the cardinality of IndexTuple established in `PrimeTensor/Depth.lean`. For the case the project actually runs on, $\mathit{dim} = \mathit{rank} =$ three, that is 27 coefficients. The index set is never empty, so a tensor always has at least one coefficient and there is no vacuous tensor to special-case.

Definition 1.3 (Vector and Matrix)**.** The abbreviations `Tensor.Vector` and `Tensor.Matrix`:

$$\text{Vector } A \mathit{\ dim} \equiv \text{Tensor } A \mathit{\ dim}\ \mathsf{one}, \qquad \text{Matrix } A \mathit{\ dim} \equiv \text{Tensor } A \mathit{\ dim}\ \mathsf{two}.$$

```
abbrev Vector (A : Type) (dim : Depth) := Tensor A dim .one
abbrev Matrix (A : Type) (dim : Depth) := Tensor A dim Depth.two
```

Both are `abbrev` and so reducible: they unfold during elaboration rather than standing as opaque definitions. This matters for the next two definitions, which would otherwise need explicit rewriting to typecheck.

1.2 Component access

Definition 1.4 (Access, `vectorAt` and `matrixAt`)**.** For v : Vector A *dim* and i : Axis *dim*, and for m : Matrix A *dim* and i, j : Axis *dim*,

$$\mathrm{vectorAt}(v, i) \equiv v.\mathrm{component}\ i, \qquad \mathrm{matrixAt}(m, i, j) \equiv m.\mathrm{component}\ (i, j).$$

```
def vectorAt {A : Type} {dim : Depth} (v : Vector A dim) (i : Axis dim) : A :=
  v.component i

/-- Rank-two component access. -/
def matrixAt {A : Type} {dim : Depth} (m : Matrix A dim)
    (i j : Axis dim) : A :=
  m.component (i, j)
```

Proposition 1.5 (Why these typecheck)**.** v.component *expects an argument of type* IndexTuple(*dim*, one) *but is given one of type* Axis *dim, and* m.component *expects* IndexTuple(*dim*, two) *but is given a literal pair. Both are accepted, with no coercion and no cast, because*

$$\mathrm{IndexTuple}(dim, \mathsf{one}) \;=\; \mathrm{Axis}\ dim, \qquad \mathrm{IndexTuple}(dim, \mathsf{two}) \;=\; \mathrm{Axis}\ dim \times \mathrm{Axis}\ dim$$

hold definitionally.

Proof. IndexTuple is defined in `PrimeTensor/Depth.lean` by structural recursion on the rank, with defining equations IndexTuple(*dim*, one) = Axis *dim* and IndexTuple(*dim*, succ r) = Axis *dim* × IndexTuple(*dim*, r). The first equation gives the rank-one case directly. For the rank-two case, two unfolds to succ one, so the second equation applies and then the first, giving Axis *dim* × Axis *dim*. Each step is a defining equation of a structural recursion at a constructor, hence a definitional equality, and Vector and Matrix are reducible so they do not block the reduction. □

Design note 1.6. This is what the choice of IndexTuple in `PrimeTensor/Depth.lean` was for. Had index tuples been an inductive family, a rank-one tuple would have been a constructor application wrapping an axis index rather than the index itself, and every component access would have had to pack and unpack it — with the packing visible in the statement of every downstream lemma. Defining the index type by recursion into Type makes the wrapper disappear before it is ever built.

1.3 Operations

Definition 1.7 (Pointwise map, `PrimeTensor.Tensor.map`)**.** For $f : A \to B$ and t : Tensor A *dim rank*,

$$\mathrm{map}(f, t) \equiv \langle\, \lambda i.\ f(t.\mathrm{component}\ i) \,\rangle \ :\ \mathrm{Tensor}\ B\ dim\ rank.$$

```
def map {A B : Type} {dim rank : Depth} (f : A → B)
    (t : Tensor A dim rank) : Tensor B dim rank :=
  ⟨fun i => f (t.component i)⟩
```

Composition with f on the coefficient side; the dimension, the rank and the index set are untouched.

Remark 1.8 (The functor laws are not stated). map is functorial — $\mathrm{map}(\mathrm{id}, t) = t$ and $\mathrm{map}(g \circ f, t) = \mathrm{map}(g, \mathrm{map}(f, t))$ — and both laws in fact hold by `rfl`, since each side reduces to the same lambda under structure and function eta. Neither is stated in the file, and no `Functor` instance is registered.

Definition 1.9 (Outer product, `PrimeTensor.Tensor.outer`)**.** For a type A with a multiplication and u, v : Vector A *dim*,

$$\mathrm{outer}(u, v) \equiv \langle\, \lambda ij.\ u.\mathrm{component}\ ij_1 \cdot v.\mathrm{component}\ ij_2 \,\rangle\ :\ \mathrm{Matrix}\ A\ \mathit{dim},$$

so that its (i, j) coefficient is $u_i \cdot v_j$.

```
def outer {A : Type} [Mul A] {dim : Depth}
    (u v : Vector A dim) : Matrix A dim :=
  ⟨fun ij => u.component ij.1 * v.component ij.2⟩
```

The two projections ij_1 and ij_2 are available for the reason given in Proposition 1.5: the argument type IndexTuple(*dim*, two) *is* a product, so it can be projected without first being taken apart.

Design note 1.10 (Only a magma is required). The instance argument is `[Mul A]` alone. No associativity, no commutativity, no unit — the same discipline as the fold in `PrimeTensor/Depth.lean`, and for the same reason: the intended coefficient carrier is a strictly positive multiplicative one, which supplies a multiplication but no additive unit to fall back on. A consequence is that outer is not symmetric, $\mathrm{outer}(u, v)$ and $\mathrm{outer}(v, u)$ being transposes rather than equal, and that there is no vector playing the role of a unit for it.

Remark 1.11 (No scalars, and no contraction here). Rank is a depth, so rank zero is unrepresentable and there is no scalar tensor: outer raises rank from one to two with nothing below it. Neither is there any operation lowering rank — contraction arrives in `PrimeTensor/Tensor/Contract.lean`, the next module. Note also that outer is stated only for vectors, not at general rank.

Summary of the correspondence

Lean declaration	Kind	Here
`PrimeTensor.Tensor`	structure	Def. 1.1
`Tensor.Vector`	abbrev	Def. 1.3
`Tensor.Matrix`	abbrev	Def. 1.3
`Tensor.vectorAt`	definition	Def. 1.4
`Tensor.matrixAt`	definition	Def. 1.4
`Tensor.map`	definition	Def. 1.7
`Tensor.outer`	definition	Def. 1.9

Every declaration of `PrimeTensor/Tensor/Basic.lean` appears above. The file contains no theorem at all — no `sorry`, no axiom, no named proposition, and nothing proved. It is seven definitions, and its content is that they elaborate. Proposition 1.5 and Remarks 1.2 and 1.8 are stated for the reader and are not declarations of the file.